

Understanding Ambiguous Instructions: Scene Graphs Contextualizing Language Model Programs

Christopher Overdiek*, Ching-Huai Chen†

*M.Sc. student in Computer Science

Rheinische Friedrich-Wilhelms-Universität Bonn, Bonn, Germany

Email: s6crover@uni-bonn.de, 3122952

†M.Sc. student in Media Informatics

RWTH Aachen University, Aachen, Germany

Email: chinghuai.chen@uni-bonn.de, 464528

Abstract—Ambiguity in natural language instructions is a well known problem that has recently been tackled by models such as CodeDiffuser. This model in particular leverages VLMs on a top level to convert ambiguity into explicit object selections. Yet its main concern is not choosing the best object available, but simply choosing an object under the umbrella of its instruction at all. We investigate their code base and several Scene Graph implementations in hopes of augmenting their already strong performing model with context aware object selection methods. Though our quantitative results are limited due to unknown issues with the CodeDiffuser implementation at our disposal, we find that its object selection is decoupled from the surrounding execution in such a way that applying our approach to a functional implementation would suggest instructional fulfillment being negatively impacted to be highly unlikely while the currently random object selection could only reasonably stand to improve.

I. INTRODUCTION

A. Background

Recent years have seen significant progress in simplifying human interaction with artificial intelligence through models like CodeDiffuser, in which Large Language Models are used to bridge the gap between ambiguous, non-specific human instruction and lower level, robust movement policies [6]. This specific model has been documented to outperform several other state-of-the-art algorithms in fulfilling vague and imprecise text-based instructions to the letter.

B. Motivation

However, we argue that CodeDiffuser concerns itself primarily with the fulfillment of its instructions, rather than with how well this was accomplished. From ancient legends like "The Golem of Prague" to modern sci-fi stories like "Space Odyssey", the recurring theme of artificial life causing havoc by fulfilling commands to the letter but not to the spirit provides numerous cautionary tales about the extremes of this disparity. On a smaller scale, the instruction to hang any mug onto any branch of a mug tree (as illustrated in Fig. 1) allows for several different permutations of how the



Fig. 1: In the task example, we aim to select a pair consisting of a mug and a branch to hang it on. This depiction of the task environment is directly taken from CodeDiffuser [6].

task could be accomplished, where any randomly selected yet technically valid option – which is how CodeDiffuser resolves such variegated scenarios [6] – could differ significantly from the course of action common sense would suggest.

We hoped that integrating Scene Graphs into the CodeDiffuser pipeline would provide it with sufficient context about the relational properties between objects to allow for action selection that consistently and more closely resembles human decision making.

II. RELATED WORK

Scene Graphs were first introduced by Johnson et al. [1] as a representation for detailed semantics in images. They provide a data structure that describes the contents of a scene, including object instances (e.g., girl, racket), attributes of objects (e.g., the girl is blonde), and relationships between objects (e.g., the girl is swinging a racket). It is these we planned to leverage for our project.

Early in development, we further considered covering dynamic environments and mobile agents. A specialized model for this task was proposed by Xie et al. [5]. Although it did not ultimately come into play in our implementation, it nevertheless deserves consideration if our idea is to be expanded in the future.

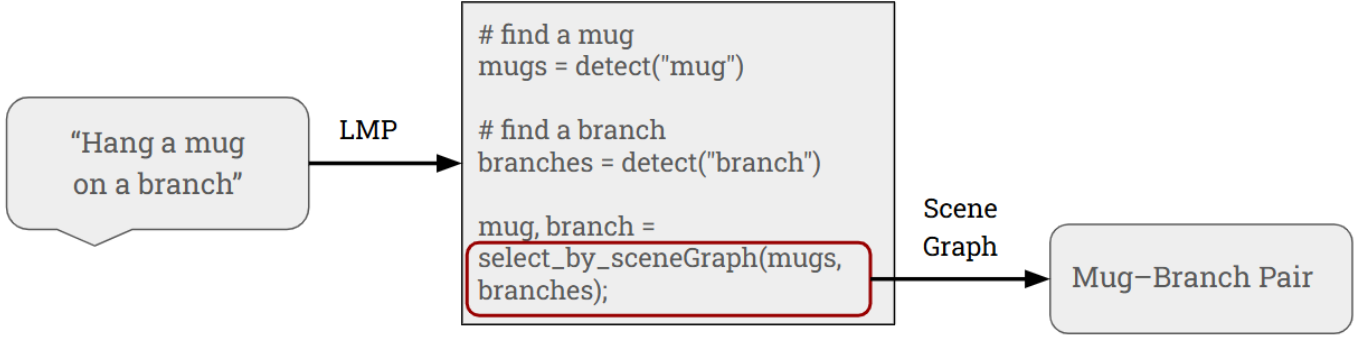


Fig. 2: Method overview. The LMP first generates executable code from the instruction and subsequently selects a target object pair using information from the scene graph.

Wald et al. [4] categorize attributes into static properties, dynamic properties, and affordances, and relationships into support relationships, proximity relationships, and comparative relationships.

Interactive properties are investigated in-depth by Zhang et al. [7], constructing Scene Graphs that capture complex scene functionalities. We deliberated implementing their model for their consideration of much more than spatial relations, but deemed our scene environment too simple to warrant such complexity.

We instead chose the Scene Graph Generation (SGG) methods proposed by Neau et al. [2] and Park et al. [3] for closer consideration. The Real-time Efficiency and Accuracy Compromise for Tradeoffs in Scene Graph Generation (REACT) architecture aims to find a balance in reaching the three objectives of improving relation prediction accuracy, enhancing object detection accuracy, and reducing latency, without neglecting any one of them. In the Synthetic Visual Genome (SVG) framework, meanwhile, the ROBIN model is proposed, which is trained on a synthetic dataset enriched with more complete object relations inferred from existing Scene Graphs by a teacher multimodal model. In addition, SVG demonstrates superior relationship understanding by applying a self-distillation strategy, in which GPT-4o is used to improve the quality of the generated Scene Graphs by filtering irrelevant relations and introducing more relevant ones.

III. METHOD

A. Approach

As our project is so closely linked to the original CodeDiffuser model, our methodology was shaped by the corresponding code base and its capabilities. Hence, we sought to first recreate the results of its research paper locally, forming a baseline for comparison of our aspired eventual modifications.

Afterwards, we decided to divide the aforementioned modifications into two separate workflows to be conducted in tandem. The first workflow focuses on Language Model Programs (LMP), as illustrated in Fig. 2, where one of us was to get acquainted with the workings of code generation via VLM so as to allow us to later inject custom code into

the code generation process. The second workflow focuses on Scene Graphs, also illustrated in Fig. 2, where the other was to seek out and, ideally, implement compatible Scene Graph implementations to provide scene information meant to augment action selection. In particular, the two most likely candidates for preexisting Scene Graph implementations were SVG [3] and REACT [2], leaving the option of choosing one over the other if performance were to vary.

Finally, and if required, we intended to implement methods to be used by CodeDiffuser’s code generation, granting easy access to non-random object selection based on the Scene Graph information. In this, our goal was to construct a pipeline that amplifies what CodeDiffuser had already achieved.

B. Hypothesis

With the original code base as the backbone, we hypothesized that our results would quantitatively match CodeDiffuser in error-free instruction fulfillment while outperforming it qualitatively in the manner of their fulfillment’s favorability. If successful up to this point, we planned to bolster evaluation of the latter with a second quantitative analysis by adding suitable numerical evaluation criteria.

IV. EXPERIMENTS

A. Implementing CodeDiffuser

It is no exaggeration to state that working with the official CodeDiffuser code base proved challenging. The installation steps provided led to a maze of library compatibility deadlocks, partially corresponding to an as-of-yet unresolved entry to the Issues tab of its GitHub repository. Unspecified libraries needed to be imported, files were missing or incorrectly referenced, local file paths needed to be generalized, and navigation of the code base meanwhile was complicated needlessly by large parts of it – largely consisting of demo files or apparent legacy code – being wholly dispensable to its intended usage. Given this, it comes as no surprise that a significant portion of development time was spent on getting CodeDiffuser to run on our systems as intended.

The key execution steps of CodeDiffuser from bare code base to final results can be split into the three steps of data

collection, training, and evaluation, each with their own main python file and a multitude of run parameters, somewhat captured in specific shell scripts in the cases of data collection and training. In the order mentioned above, we worked on getting each to execute correctly, all to apparent success. The fact that results of the evaluative step were nowhere close to desired was expected during debugging, as training cycles and dataset sizes were kept small for faster execution. Only after a full, regular execution of the full pipeline resulting in similarly poor results (see Fig. 3) did we realize that our version of the code base must have some inherent flaw so far obscured by simple crashes. What this flaw may be is still unknown to us, and its presence severely obstructed our plans for quantitative evaluation of any subsequent step.

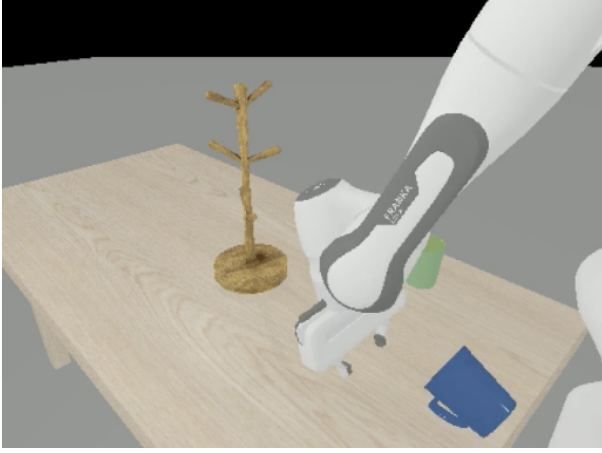


Fig. 3: Simulation of fully trained local CodeDiffuser given task of hanging mug, though currently punting it off of table instead

B. Understanding Language Model Programs

Language Model Programs, henceforth referred to as LMP's, are what CodeDiffuser utilizes to generate code snippets specific to the given text based instructions [6]. As with the implementation of CodeDiffuser itself, dissection of the LMP functionality woven into its architecture was complicated by CodeDiffuser's nebulous file hierarchies. Toward the beginning of development, when our debugging tools were still struggling to provide deeper insight into exactly which files and methods were accessed upon execution due to crashes occurring before even the first method call, we inferred that the subdirectory `language_model_program` must hold the answers we were seeking. We later learned that this was merely a vestige left over from development and that the LMP files actually used by CodeDiffuser were buried deep within `d3fields_dev`, yet the former's workings were generally the same and equally if not more complex, so our gleaned insights transfer nicely.

LMP's function by supplying a VLM with a tuple of prompts in string or `.txt`-file format and expecting a response resembling an executable code snippet which can then be

executed. This relation can be represented as:

$$\text{LMP} : (q_0, q, P) \mapsto \pi \sim \Delta(\Pi) \quad (1)$$

where:

$$q_0 : \text{Setup query for VLM} \quad (2)$$

$$q : \text{Instruction for code generation} \quad (3)$$

$$P : \text{Set of prompt files} \quad (4)$$

$$\pi : \text{Generated code snippet} \quad (5)$$

$$\Delta(\Pi) : \text{Distribution over all viable code snippets} \quad (6)$$

More specifically, the tuple (q_0, q, P) generates a randomly sampled code snippet π from within the distribution of code snippets $\Delta(\Pi)$; a distribution from which each snippet would attempt to follow instruction q in its own way. The set q_0 instructs the VLM on its task of generating nothing but a code snippet based on whatever q demands to be done, q contains the ambiguous instruction passed to the pipeline and P represents some set of prompts for reference in code generation. These prompts are `.txt` files containing example code snippets of some arbitrary robot tasks, where the topmost comment describes the example instance's instruction and the following snippet first disassembles the instruction into string representations of the objects of interest and then uses them in specific methods for object detection and action sequence generation.

The broader formatting of these prompts P is of little relevance, with the exception of some key concerns. It must for one be ensured that the files' syntax complies with the programming language meant to execute the resultant output. In effect, the prompt must constitute a hypothetically properly executable representation of its instruction. Additionally, any and all methods used in the prompts must be real, callable and within reach of where the generated code is to be executed. This notably makes augmenting the code generation process with custom methods an easy task, simply requiring modifying the prompts P to correctly use the desired methods and ensuring their callability upon executing the generated code snippet π .

C. Scene Graph Generation

In trying to decide between SVG [3] and REACT [2], we initially experimented with the latter of the two models. However, REACT fails to detect branch objects and does not allow customization of the target object set, as object features are tightly coupled with the Scene Graph generation process during object detection. Consequently, this method cannot be directly applied to scenarios requiring customized object definitions. So SVG proved itself as the clear winner through the process of elimination.

First, since we found that some existing detection algorithms (such as YOLOv11) cannot reliably detect branches which are themselves only part of a larger construct, we also examined how CodeDiffuser handles branch detection through additional reference annotations. Based on this, we decided to manually label the locations of the branches and generate masks for

different branch regions. Our Scene Graph generation pipeline first extracts masks for all objects in the scene. From these candidates, we manually select the relevant objects, including two mugs, four branches we manually labeled, and one branch base, and feed them into the next stage to construct the Scene Graph, as illustrated in Fig. 4.

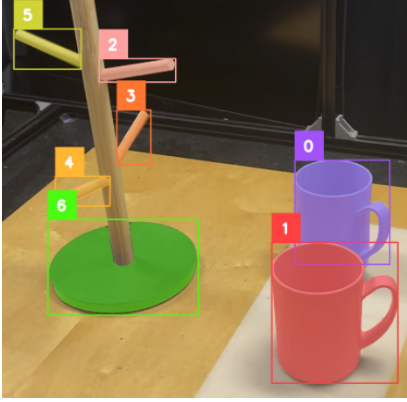


Fig. 4: Annotated illustration based on Fig. 1, highlighting the relevant task objects.

Due to the involvement of VLM-based reasoning in the Scene Graph generation process, the generated results are not deterministic and may vary across different runs. Therefore, we repeated the generation process 100 times and collected the resulting Scene Graphs in JSON format to provide a baseline for evaluating our implementation’s performance. The extracted attribute distributions are summarized in Tables I and II.

With functional Scene Graphs and their corresponding images, we next planned to provide them to the VLM, which would select the optimal object pair based on both visual information and structured relational representations.

By way of example, in one instance of the simple test case of the instruction “hang a mug on a branch,” the VLM selected the target mug and branch based on the two relations “the blue mug is attached to the right branch” and “the right branch is supporting the blue mug,” demonstrating how structured relational information guides its decision.

D. Results

In this fourth and final step of development, we envisioned merging the two branches of LMP modification and Scene Graph generation into the preexisting CodeDiffuser code base. The resulting model would have allowed us to experiment with the expanded contextual awareness supplied by Scene Graph and what potential object selection prioritization this could allow us, backed by the quantitative evaluation included in CodeDiffuser. We did not anticipate that merely implementing CodeDiffuser as the very cornerstone of this process would turn out to be the failure point.

That being said, our experiments and experiences in those areas unrestricted by the failure of the core code base left us

with plenty of other results to consider for an estimate of what could have been expected in the ideal scenario.

In dissecting LMP, we reached a deeper understanding of the task at hand. The current random elements in CodeDiffuser’s object selection appear to be less an explicit choice of implementation and more a byproduct of leaving object selection within the bounds of successful instruction fulfillment up to an inherently stochastic VLM by not specifying any external methods to be used in picking from among several equally viable objects. Then it should stand to reason that implementing some such methods is likely to instantly upgrade the LMP’s capability for sensible object selection, as it is impossible to perform worse than random choice for our purposes. At the same time, the risk for diminishing its performance instead is very low, as failure points are restricted to clashes between our new object selection methods and the preexisting methods, which can again be reasonably disregarded due to how any custom methods would effectively be substituted into the current implicit random object selection and would as such be near to wholly independent from the remainder of the execution.

TABLE I: Summary of attributes and relations of each mug across 100 generations. Each category represents a semantic group, such as color (COL), handle-related attributes (HND), and relational properties (REL). The reported percentages indicate the relative frequencies, after accounting for synonyms, belonging to each category.

Category	Attribute	Blue mug	Pink mug
COL	Shades of Blue	86%	0%
	Shades of Pink	1%	98%
	Other Colors	12%	0%
	Unspecified	1%	2%
HND	Some Handle	13%	14%
	Colored Handle	6%	9%
	Unspecified	81%	77%
REL	Above Base	2.35%	0%
	In Front of Base	5.88%	11.9%
	Supported by Base	7.06%	7.14%
	Other Rel. to Base	4.71%	2.38%
	Supported by any Branch	5.88%	3.57%
	In Front of Other Mug	0%	10.71%
	Behind Other Mug	10.59%	0%
	Next To Other Mug	57.65%	59.52%
	Other Rel. To Other Mug	5.88%	4.76%

Another ray of hope is provided by our results in implementing Scene Graph. Both Tables I and II suggest that this implementation, at the very least, is perfectly functional. All attributes that could be expected are appropriately dominantly represented; The slight positional bias of category POS in Table II towards the Top Left is easily explained by this being the de facto location of the mug tree itself relative to the scene as a whole. At the same time, some exploration is conducted by the stochastic workings of SVG’s SGG [3]. The task environment of hanging one of two mugs onto one of four branches naturally does not necessitate particularly

pedantic attribute generation, but the apparent capability for exploration of our implementation and certainly the paper behind the model itself suggest that more complex scenarios would promote diversification in the attributes at our disposal.

TABLE II: Summary of key attributes of each branch (Top Right, Bottom Right, Bottom Left and Top Left) across 100 generations. Represented are type of object (TYP) and relative position (POS). For the latter, semantic representations have been grouped once by vertical (e.g. "Top Right" and "Top Left" both contribute to "Top Row") and once by horizontal (e.g. "Top" and "Bottom" both contribute to "Middle Column") connotation.

Category	Attribute	TR	BR	BL	TL
TYP	Arm	9%	6%	6%	8%
	Handle	12%	11%	12%	11%
	Hook	22%	21%	22%	21%
	Peg	5%	5%	5%	6%
	Stick	28%	31%	29%	29%
	Other Types	20%	22%	24%	18%
	Unspecified	4%	4%	2%	7%
POS	Top Row	37%	11%	5%	46%
	Middle Row	31%	52%	39%	14%
	Bottom Row	0%	0%	20%	1%
	Left Column	13%	13%	47%	24%
	Middle Column	15%	45%	11%	31%
	Right Column	40%	5%	6%	6%
	Unspecified	32%	37%	36%	39%

V. DISCUSSION & CONCLUSION

A. Discussion

Under the CodeDiffuser framework, random object selection is insufficient for producing consistently optimal decisions where optimality is defined by deeper comprehension of the task than its literal definition. In contrast, incorporating Scene Graphs could provide richer attribute information of objects and relational information, which might be leveraged by VLMs to support more informed pair selection. It is regrettable, then, that a quantitative evaluation of one such implementation was denied us. But our results can paint a picture of our approach's potential nonetheless.

As previously stated, CodeDiffuser's usage of LMPs nicely decouples object selection from all other elements of its execution by foisting it off onto an implicit, randomized selection by VLM code generation. As such, the addition of more sophisticated selection methods poses no issue other than forcing the LMP into actually using them, which is easily achieved.

Additionally, our implementation of SVG works as intended and their VLM generated attributes feed into LMPs nicely. Whether a single SGG is used or a database is built from merging numerous generations into one large attribute and relation look-up set, the additional context granted by SVG's promises countless ways in which calculated object selection could be achieved even in an ambiguous setting.

B. Conclusion

Nevertheless, several limitations remain, even beyond the most damning fact that our lack of a functional CodeDiffuser implementation restricts us to estimation and postulation.

First, branch locations are currently labeled manually, which limits scalability to larger and more diverse datasets.

Second, the use of single-image inputs restricts depth and spatial reasoning, such as in identifying the closest target in the scene.

Third, the variety of attributes for contextual object selection scales directly with complexity of the scene, so an overly simplistic scene is a poor environment for the development of object selection criteria.

Finally, and though the current Scene Graph generation relying on VLM-based reasoning feeds into LMP's quite nicely, it must be said that the generated explanations are not always sufficiently justified. In some cases, the underlying rationale behind object compatibility remains unclear and requires further validation and refinement (for example, the reason why the right branch is considered to support the blue mug in our test case).

So while our results are promising, they are still to be taken with a grain of salt until anything can be truly verified in experimentation, and our project's limited scope due to numerous restrictions in development requires significant expansion if it is ever to find application in anything but highly specific use-cases, which mirrors what CodeDiffuser already set out to do.

C. Future Work

Building on these goals, future work might focus on addressing these limitations by improving automatic annotation, incorporating multi-view perception, expanding the testing environment, and enhancing the reliability and interpretability of Scene Graph reasoning. These augmentations would finally match our aspirations of a flexible yet robust artificial intelligence that still manages to read between the lines. Though this clearly still depends on using a functional implementation of CodeDiffuser at its core, so for any future work, that might just be the place to start.

REFERENCES

- [1] Justin Johnson, Ranjay Krishna, Michael Stark, Li-Jia Li, David Shamma, Michael Bernstein, and Li Fei-Fei. Image retrieval using scene graphs. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3668–3678, 2015.
- [2] Maëllic Neau, Paulo E Santos, Anne-Gwenn Bosser, Cédric Buche, and Akihiro Sugimoto. React: Real-time efficiency and accuracy compromise for tradeoffs in scene graph generation. *arXiv preprint arXiv:2405.16116*, 2024.
- [3] Jae Sung Park, Zixian Ma, Linjie Li, Chenhao Zheng, Cheng-Yu Hsieh, Ximing Lu, Khyathi Chandu, Quan Kong, Norimasa Kobori, Ali Farhadi, et al. Synthetic Visual Genome. In *Proceedings of the Computer Vision*

and *Pattern Recognition Conference*, pages 9073–9086, 2025.

- [4] Johanna Wald, Helisa Dharmo, Nassir Navab, and Federico Tombari. Learning 3d semantic scene graphs from 3d indoor reconstructions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3961–3970, 2020.
- [5] Fujing Xie, Sören Schwertfeger, and Hermann Blum. osmAG-LLM: Zero-Shot Open-Vocabulary Object Navigation via Semantic Maps and Large Language Models Reasoning, 2025. URL <https://arxiv.org/abs/2507.12753>.
- [6] Guang Yin, Yitong Li, Yixuan Wang, Dale McConachie, Paarth Shah, Kunimatsu Hashimoto, Huan Zhang, Katherine Liu, and Yunzhu Li. CodeDiffuser: Attention-Enhanced Diffusion Policy via VLM-Generated Code for Instruction Ambiguity. In *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [7] Chenyangguang Zhang, Alexandros Delitzas, Fangjin-hua Wang, Ruida Zhang, Xiangyang Ji, Marc Pollefeys, and Francis Engelmann. Open-Vocabulary Functional 3D Scene Graphs for Real-World Indoor Spaces. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.